Software Reengineering

Mids Notes | Bilkul aasan Roman Urdu mein, samjha kar | Har topic ke neeche paper mein likhne wala jawab

Kaise parhna hai: Har topic mein pehle main aap ko **kahani ki tarah samjhata hoon** (Roman Urdu mein, ratta nahi). Phir **peela box** mein ek asaan example hai. Aakhir mein **neela box** hai jis mein 3 se 6 chhote English jumle hain: yeh woh jawab hai jo aap **paper mein seedha likh sakte ho**. Pehle samjho, phir neela box parh lo.

Dhyan: is subject ki slides ya book nahi mili, is liye yeh standard textbooks ki maloomat se hai. Paper se pehle sir ki slides se naam aur tarteeb ek baar milaan kar lena.

Chapter 1: Legacy Software System

1.1 Legacy system kya hota hai?

Socho aap ke dada jaan ke zamane ka ek bohat purana scooter hai. Woh abhi bhi chalta hai, aur ghar ke liye bohat zaroori bhi hai. Magar uske spare parts nahi milte, mistri usay theek karne se darte hain, aur kisi ko pata nahi ke andar kya kya jura hua hai. Software mein bhi aisa hota hai. Jo **purana software** abhi tak chal raha hai, business ke liye bohat zaroori hai, magar purani technology par bana hone ki wajah se usay badalna aur samajhna mushkil hai, usay **legacy system** kehte hain.

Legacy ka matlab hai 'virasat', jo purane zamane se humein mili. Yeh nayi cheez nahi, magar hum isay chhor bhi nahi sakte.

Example: Bank ka woh software jo 30 saal pehle COBOL language mein bana tha aur aaj bhi roz lakhon logon ke paise ka hisaab rakhta hai. Isay banane walay retire ho chuke hain, magar bank isay band nahi kar sakta kyunke poora bank isi par chalta hai.

Paper mein aise likho:
A legacy system is an old software system built with outdated technology that is still used because it performs critical business functions. It is usually hard to understand, modify and maintain. Example: an old bank system written in COBOL running on a mainframe.

1.2 Legacy systems ki khasiyatein (pehchan)

Legacy system ko pehchanne ke liye yeh baatein dekho. Pehli baat, yeh **bohat purana aur bohat bara** hota hai, lakhon lines ka code. Doosri, yeh **business ke liye bohat zaroori** hota hai, band hua to kaam ruk jata hai. Teesri, iski **documentation** (likhi hui maloomat) ya to hai hi nahi ya purani aur ghalat hai. Chauthi, jin logon ne isay banaya woh ab company mein nahi, to system ka ilm unke saath chala gaya. Paanchwi, yeh **purani languages aur purane hardware** par bana hota hai. Chhatthi, isme **koi bhi change karna mehnga aur khatarnak** hota hai: ek jagah badlo to doosri jagah kuch toot jata hai. Aur saatwi, baar baar patch lagne se iska code **uljha hua (spaghetti code)** ban jata hai.

Paper mein aise likho:
Characteristics of legacy systems: old and large, business-critical, poor or missing documentation, original developers have left, outdated technology, difficult and costly to change, and tangled (spaghetti) code structure.

1.3 Legacy system phir bhi kyun chalta rehta hai?

Aap soch rahe hoge ke itna purana hai to naya kyun nahi bana lete? Wajah yeh hai ke naya system banana **bohat mehnga aur bohat risky** hai. Agar naye system mein ek bhi ghalti reh gayi to pura business ruk sakta hai. Phir, purane code ke andar saalon ki **business rules** chhupi hui hain (jaise tax kaise lagta hai, kis customer ko discount milta hai) jo kisi ko poori tarah yaad nahi. Naya banate waqt woh bhool gaye to nuqsan. Isi liye company kehti hai 'jab tak chal raha hai, chalne do'.

Paper mein aise likho:
Legacy systems stay in use because replacing them is costly and risky, they contain valuable business rules built up over many years, and the business depends on them every day.

1.4 Legacy systems ke masle

1) Maintenance bohat mehnga. Chhota sa change karne mein hafte lag jate hain, kyunke code uljha hua hai. **2) Documentation nahi.** Naya developer poochta hai 'yeh function kya karta hai?' aur jawab dene wala koi nahi. **3) Skilled log kam.** COBOL jaise purani language jaanne wale kam aur mehnge hain. **4) Naye systems se jurna mushkil.** Purane bank software ko mobile app se connect karna aasan nahi. **5) Purana hardware.** Machine kharab ho to uske parts nahi milte. **6) Security kamzor.** Purane software mein aaj ke hackers ke khilaf hifazat nahi. **7) Software erosion.** Waqt ke saath baar baar ki tabdeeliyon se software ka structure bigarta jata hai, jaise purani imarat jis mein jagah jagah jodh toar ho.

Paper mein aise likho:
Problems of legacy systems: high maintenance cost, missing documentation, shortage of skilled staff, difficult integration with new systems, obsolete hardware, weak security, and gradual decay of the software structure.

1.5 Legacy system ke hisse

Legacy system sirf code ka naam nahi. Isme **chhe cheezein** hoti hain. (1) **Hardware**, jis machine par chalta hai. (2) **Support software**, jaise operating system aur compiler. (3) **Application software**, asal programs. (4) **Application data**, database aur files. (5) **Business processes**, kaam karne ke tareeqe jo system se chalte hain. (6) **Business policies aur rules**, company ke usool jo code ke andar chhupe hain.

Paper mein aise likho:
A legacy system consists of hardware, support software, application software, application data, business processes and business policies and rules.

1.6 Legacy system ka kya kiya jaye? (4 raaste)

Management ke saamne 4 raaste hote hain. Faisla karne ke liye do sawal poochte hain: system **business ke liye kitna zaroori** hai (business value), aur woh **andar se kitna achha** hai (technical quality)?

1) Scrap (band kar do): agar koi use hi nahi karta, to bas band kar do. **2) Maintain (jaisa hai chalne do):** agar zaroori bhi hai aur achha bhi, to sirf thori maintenance. **3) Reengineer (andar se sudharo):** agar zaroori hai magar andar se kharab hai, to usay dobara sudhaar kar naya jaisa bana do. **4) Replace (naya banao):** agar technology itni kharab hai ke sudharna mumkin hi nahi, to bilkul naya system banao.

Example: Ek purana 'Student Fee' software hai. Har roz use hota hai (zaroori) magar uska code bohat uljha hua hai (kharab quality). To hum usay band nahi karenge, balke reengineer karenge.

Paper mein aise likho:
There are four options for a legacy system: scrap it, keep maintaining it, reengineer it, or replace it with a new system. If the system has high business value but poor technical quality, reengineering is the best option.

Chapter 2: Software Reengineering

2.1 Software Reengineering kya hai?

Ek purani imarat hai jo dheere dheere kharab ho rahi hai. Aap do kaam kar sakte ho: usay **gira kar nayi bana do** (mehnga), ya usay **renovate** kar do: bunyaad aur kamray wohi, magar andar se naya rang, nayi wiring, nayi pipes. Software mein isay **reengineering** kehte hain. Yaani purane software ko **dobara tayyar karna** taake woh behtar, saaf aur aaj ke zamane ke qabil ho jaye, magar uska **kaam (functionality) wohi rahe**.

Reengineering ke 3 hisse hote hain, jo aap ko yaad rakhne hain: pehle purane code ko **samajhna** (Reverse Engineering), phir usay **saaf karna** (Restructuring), phir naye tareeqe se **dobara banana** (Forward Engineering).

Example: Purana 'Result' software jo uljhe code mein hai. Reengineering mein pehle us ka code parh kar samjha (reverse), phir code saaf kiya (restructure), phir naye framework mein dobara bana diya (forward). Student ke liye result dekhne ka tareeqa wohi, magar andar se system saaf aur tez.

Paper mein aise likho:
Software reengineering is the examination and alteration of an existing system to reconstitute it in a new form, and the implementation of that new form. It improves quality and maintainability while keeping the same functionality. It includes reverse engineering, restructuring and forward engineering.

2.2 Reengineering kyun karte hain?

Pehli wajah **cost** hai: naya system banana bohat mehnga hai, reengineering sasta hai. Doosri wajah **risk** hai: purana system chalta hua hai to us ke andar ka business knowledge mehfooz rehta hai, naya bana kar kuch bhoolne ka darr nahi. Teesri wajah **maintainability**: saaf code ko samajhna aur badalna aasan hota hai. Chauthi wajah **nayi technology**: purane software ko web, mobile ya cloud par le jana. Paanchwi wajah **documentation**: jo kho gayi thi woh dobara ban jati hai.

Paper mein aise likho:
Reasons for reengineering: lower cost and risk than rewriting, preserving existing business knowledge, better maintainability, moving to new technology, and recovering lost documentation.

2.3 Maintenance, Reengineering aur Naya system mein farq

Gari ki misaal lo. **Maintenance** matlab chhoti servicing: tyre badal diya, oil change. **Reengineering** matlab gari ka engine khol kar poora sudharna, magar gari wohi. **Replacement (naya system)** matlab purani gari bech kar nayi gari kharidna. Cost aur risk: maintenance mein sab se kam, reengineering mein darmiyana, naya system banane mein sab se zyada.

Paper mein aise likho:
Maintenance makes small changes after delivery, reengineering restructures a large part of the system without changing its function, and replacement builds a completely new system. Cost and risk are lowest for maintenance and highest for replacement.

2.4 Zaroori Terms: Forward, Reverse, Restructuring waghera

Yeh terms sab se zyada paper mein poochay jate hain aur sab se zyada confuse karte hain. Ek ghar ka example lo:

Forward Engineering: naksha banao, phir ghar banao. Yaani design se code ki taraf. Yeh normal tareeqa hai nayi cheez banane ka.

Reverse Engineering: ghar pehle se bana hua hai magar naksha kho gaya. Aap ghar ko dekh kar uska naksha dobara banate ho. Yaani **code se design** ki taraf. Isme kuch badalta nahi, sirf **samajh kar likhte hain**.

Redocumentation: reverse engineering ka ek hissa. Purani documentation ya comments dobara likhna, jaise ghar ke naksha ki nayi copy.

Design Recovery: code aur kaam ki maloomat se yeh pata lagana ke **design kya tha aur kyun tha**.

Restructuring: ghar ke andar saman ki tarteeb saaf karna. Deewarein wohi, kaam wohi, bas andar ka structure saaf. Isme **behavior nahi badalta**.

Reengineering: upar ke sab ka majmua. Pehle reverse, phir restructure, phir forward.

Paper mein aise likho:
Forward engineering moves from design to code. Reverse engineering analyzes existing code to recover its design and specification, without changing it. Restructuring transforms the structure of a system at the same abstraction level without changing its behavior. Reengineering combines reverse engineering, restructuring and forward engineering.

2.5 Reengineering ki 2 strategies

Big-bang (ek saath): poore system ko ek hi dafa reengineer kar ke naya chala dena. Jaldi poora naya system mil jata hai, magar agar kuch ghalat ho gaya to sab ruk jata hai, is liye risky hai. **Incremental (hissa hissa):** system ke chhote chhote hisse ek ek kar ke reengineer karna, aur purana aur naya kuch der saath chalte hain. Isme risk kam hai aur ghalti jaldi pakri jati hai, magar purane aur naye ko jorne ka extra kaam hota hai.

Example: Ghar ka renovation: ek saath poora ghar khol dena (big-bang) ya ek ek kamra theek karna (incremental). Doosre mein aap ghar mein rehte bhi ho sakte ho.

Paper mein aise likho:
In the big-bang strategy the whole system is reengineered at once, which is fast but risky. In the incremental strategy the system is reengineered part by part, which is safer because errors are found early, but needs extra effort to connect old and new parts.

2.6 Reengineering ke fayde aur nuqsan

Fayde: naye system se sasta aur kam risky; business knowledge mehfooz; maintainability aur quality behtar; nayi technology ka raasta khulta hai. **Nuqsan:** agar design bohat hi kharab ho to sirf reengineering se hal nahi hota; waqt aur mehnat bohat lagti hai; purani architecture ki kuch hadein poori tarah nahi mit pati.

Paper mein aise likho:
Advantages: cheaper and less risky than new development, keeps business knowledge, improves quality. Disadvantages: needs time and effort, and cannot fix every limit of a very poor design.

Chapter 3: Software Evolution and Maintenance

3.1 Software Maintenance kya hai?

Software bante hi khatam nahi hota. Jaise gari ko khareedne ke baad bhi servicing, tyre aur petrol chahiye, waise software ko **user ko dene (release) ke baad** bhi dekh bhaal chahiye. Isi dekh bhaal ko **maintenance** kehte hain. Software mein roz naye masle aate hain: koi bug mila, naya phone aaya, user ne naya feature maanga. Yeh sab maintenance ka hissa hai.

Paper mein aise likho:
Software maintenance is the modification of a software product after delivery to correct faults, improve performance or other attributes, or adapt it to a changed environment.

3.2 Maintenance ki 4 qismein

1) Corrective maintenance (ghalti theek karna): app crash ho rahi thi, user ne bataya, aap ne bug theek kiya. Yeh 'ilaaj' hai, jo kharab ho gaya usay theek karna.

2) Adaptive maintenance (maahol ke mutabiq badalna): software ke aas paas ki cheezein badal gayin: naya Android version aa gaya, ya sarkar ne tax rate badal diya. Software ko us naye maahol mein chalne ke liye badalna adaptive hai.

3) Perfective maintenance (behtar banana): app theek chal rahi hai, magar user kehta hai search tez ho ya naya report chahiye. Isay behtar banana perfective hai. Aam tor par sab se zyada kaam (taqreeban aadha) yehi hota hai.

4) Preventive maintenance (pehle se bachao): abhi koi masla nahi, magar code uljha hua hai aur aage masle aa sakte hain. Isliye pehle hi usay saaf kar dena (jaise refactoring). Yeh 'vaccine' hai. Yeh sab se kam kaam hota hai, magar bohat faida ka hai.

Example: Ek mobile wallet app: crash theek karna = corrective; naye Android 16 par chalana = adaptive; naya 'bill pay' feature = perfective; purane messy code ko saaf karna = preventive.

Paper mein aise likho:
Types of maintenance: Corrective fixes faults. Adaptive changes the software for a new environment such as a new OS or law. Perfective adds features or improves performance. Preventive improves the structure to avoid future problems.

3.3 Software Evolution kya hai?

Evolution ka matlab hai **waqt ke saath badalte aur barhte rehna**. Jaise insaan bachpan se jawani tak badalta hai, waise software ki zindagi mein bhi users ki nayi mangon aur nayi technology ke mutabiq musalsal badlao aata hai. Maintenance us badlao ke **kaam** hain, aur evolution us badlao ka **poora silsila**.

Paper mein aise likho:
Software evolution is the continuous process of changing and growing a software system over its life to meet changing needs and technology.

3.4 Lehman ke 8 Laws (aasan misalon ke saath)

Manny Lehman ne bade software ko dekh kar yeh usool nikale ke software waqt ke saath kaise behave karta hai:

1) Continuing Change: software ko badalte rehna parta hai, warna woh dheere dheere bekaar ho jata hai. (Purani app jo naye phone par na chale.)

2) Increasing Complexity: jaise jaise badlao hote hain, code uljhta jata hai, jab tak koi usay saaf karne ki koshish na kare. (Patch par patch lagne se code spaghetti ban jata hai.)

3) Self-Regulation: software ki evolution ki apni ek raftar aur pattern hota hai jo khud bana leta hai. (Har release mein lagbhag ek jitni tabdeeliyan.)

4) Conservation of Organisational Stability: team ke kaam ki raftar lagbhag ek jitni rehti hai, chahe aap kitne hi aur developers daal do. (Zyada log daalne se kaam hamesha dugna nahi hota.)

5) Conservation of Familiarity: har release mein tabdeeli ek hadd tak hi ho sakti hai, warna team aur users system ko samajh nahi pate. (Ek hi baar mein poora interface badla to users uljh jate hain.)

6) Continuing Growth: users ki mangon ke saath software mein features barhte rehte hain. (WhatsApp mein roz naye features.)

7) Declining Quality: agar software ko actively sudhara na jaye to uski quality dheere dheere girti jati hai. (Purane patch, purane masle.)

8) Feedback System: evolution ek feedback loop hai: users ki feedback se tabdeeli, aur tabdeeli se nayi feedback. (App store reviews dekh kar update.)

Paper mein aise likho:
Lehman's laws describe how software changes over time: continuing change, increasing complexity, self-regulation, conservation of organisational stability, conservation of familiarity, continuing growth, declining quality and feedback system. For example, under the law of continuing change, software that is not updated becomes less useful.

3.5 Maintenance mehnga kyun hota hai?

Pehli wajah **team badalna**: jo log system jaante the woh chale gaye, naye logon ko seekhna parta hai. Doosri, maintenance karne wale aksar **kam tajurba** wale hote hain. Teesri, program jitna **purana aur uljha** ho, utna mehnga. Chauthi, **documentation ki kami**. Paanchwi, **ripple effect**: ek jagah chhota change kiya, to uska asar doosri jagahon par phail kar naye bugs bana deta hai (jaise paani mein kankar phenko to lehrein door tak jati hain). Isi liye change karne se pehle **impact analysis** karte hain, yaani dekhte hain ke kaun kaun si cheezein asar mein aayengi.

Paper mein aise likho:
Maintenance is expensive because of team instability, lack of staff experience, old and complex programs, poor documentation, and the ripple effect, where a change in one place causes errors elsewhere.

3.6 Maintenance process (qadam ba qadam)

Pehle koi **change request** aati hai (user ya developer ki taraf se). Phir **impact analysis** hota hai ke change se kya kya asar mein aayega. Phir **planning aur design**, phir **code mein tabdeeli (implementation)**. Uske baad **regression testing**: check karte hain ke purani cheezein toot to nahi gayin. Aakhir mein **release aur documentation update**.

Paper mein aise likho:
The maintenance process: change request, impact analysis, planning and design, implementation, regression testing, and release with updated documentation.

3.7 Software ki zindagi ke stages

Software ki zindagi ke 5 stages samjho: pehle **Initial development** (pehla version banta hai). Phir **Evolution** (naye features aasani se add hote hain). Phir **Servicing** (ab sirf chhote bug fix hote hain, kyunke structure kharab ho chuka). Phir **Phase-out** (naye changes band, replace ki tayyari). Aakhir mein **Close-down** (system band). Reengineering ka faida yeh hai ke jab software servicing ke qareeb ho, to reengineering usay dobara **evolution** wali achhi halat mein la sakti hai.

Paper mein aise likho:
Software life stages: initial development, evolution, servicing, phase-out and close-down. Reengineering can bring a system in the servicing stage back to the evolution stage.

Chapter 4: Software Reengineering Process Model

4.1 Reengineering process model kya hai?

Reengineering ek din ka kaam nahi. Yeh kayi qadam hain jo ek tarteeb se kiye jate hain, isay **process model** kehte hain. Pressman ke mutabiq yeh **6 activities ka cycle** hai. Cycle isliye ke agar kisi qadam par masla aaye to peechay ja kar dobara kar sakte hain.

6 activities ki tarteeb yaad rakho: Inventory Analysis -> Document Restructuring -> Reverse Engineering -> Code Restructuring -> Data Restructuring -> Forward Engineering.

Isay samajhne ke liye ek kahani lo: ek company ke paas bohat se purane software hain, aur humein ek purana 'School Management' software reengineer karna hai.

Qadam 1: Inventory Analysis (list banao)

Sab se pehle company ke **saare software ki list (inventory)** banate hain, aur har ek ke bare mein likhte hain: kitna purana hai, kitna bara hai, kitna zaroori hai, kitna kharcha karta hai, kitni baar kharab hota hai. Phir faisla karte hain ke **kis ko pehle sudharna hai** (jo zaroori bhi ho aur jis ka maintenance sab se mehnga ho). Kahani mein: 20 software mein se School Management ko chuna kyunke 18 saal purana hai aur har mahine 40 bug deta hai.

Paper mein aise likho:
Inventory analysis: prepare a list of all existing applications with details such as age, size, business importance and maintenance cost, and then select the best candidates for reengineering.

Qadam 2: Document Restructuring (documentation theek karo)

Purane software ki documentation ya to hoti nahi ya ghalat hoti hai. Yahan **teen options** hote hain: (1) **kuch na karo** agar system stable hai aur badalna nahi, kyunke har cheez ki documentation banana mehnga hai. (2) **Sirf us hisse ki documentation banao jo badal raha hai**. (3) **Poori documentation dobara banao** agar system business ke liye bohat zaroori hai.

Paper mein aise likho:
Document restructuring deals with weak documentation. The options are: do nothing if the system is stable, document only the parts that change, or fully redocument a business-critical system.

Qadam 3: Reverse Engineering (samjho)

Ab hum purane code ko **parh kar samajhte hain**: data kaise store hota hai, program ka structure kya hai, har hissa kya karta hai. Tools ki madad bhi lete hain. Is qadam mein **kuch naya nahi banta, sirf samajh aata hai** aur likha jata hai. Design recovery aur redocumentation yahin hoti hain.

Paper mein aise likho:
Reverse engineering analyzes the existing program to recover its design, data structures and functionality. Nothing is changed; the aim is to understand the system.

Qadam 4: Code Restructuring (code saaf karo)

Ab hum us code ko saaf karte hain jo uljha hua hai. Lambe functions ko chhota karte hain, GOTO hatate hain, nested conditions saaf karte hain. Kaam wohi rehta hai, magar code parhna aur badalna aasan ho jata hai. Baad mein test karte hain ke koi nayi ghalti to nahi aayi.

Paper mein aise likho:
Code restructuring analyzes the source code and rewrites unstructured parts into a clean, well-structured form without changing behavior. The result is reviewed and tested.

Qadam 5: Data Restructuring (data saaf karo)

Kabhi kabhi sirf code nahi, **data ka design** bhi kharab hota hai. Jaise ek hi student ka naam teen tables mein teen tarah likha hai (Ali Khan, A. Khan, Khan Ali). Data restructuring mein pehle maujooda data structures ko samajhte hain, phir unhein saaf, standard aur ek jagah kar dete hain, aur kharab files ko achhe database mein badal dete hain. Yeh bhi ek bara reengineering kaam hai.

Paper mein aise likho:
Data restructuring cleans and reorganizes the data architecture. Existing data structures are analyzed, then data models, names and formats are standardized, for example by moving old flat files into a well-designed database.

Qadam 6: Forward Engineering (naya banao)

Aakhri qadam: reverse engineering se jo design aur maloomat mili unhein istemal kar ke **naya, behtar system banate hain**, jaise naye platform ya framework par. Isay **reclamation** bhi kehte hain, yaani purane software se qeemti cheez (business logic) nikal kar naye mein daalna. Yahan naye features bhi mil sakte hain.

Paper mein aise likho:
Forward engineering uses the information gained from reverse engineering to build a new and better system, often on a new platform, reusing the valuable business logic of the old one.

4.2 Doosra model (Sommerville): ek nazar

Agar sir ne Sommerville wala model parhaya ho to uske 5 qadam yeh hain: **Source code translation** (purani language se nayi language mein code badalna), **Reverse engineering** (samajhna), **Program structure improvement** (structure saaf karna), **Program modularisation** (related hisson ko ek jagah jama karna), aur **Data reengineering** (data saaf karna). Dono models lagbhag ek jaisi cheezein hain, bas naam aur tarteeb thori alag.

Khud karne wale 9 Topics

Topic 1: Software Restructuring

Restructuring ka matlab hai software ka **structure behtar banana, magar bahar se uska kaam bilkul na badalna**. Misal: aap apne kamray mein saman idhar udhar pada hai. Aap sab kuch tarteeb se rakh dete ho. Kamray ki deewarein wohi, kaam wohi, bas ab raasta saaf aur cheezein dhoondna aasan. Software restructuring bhi aisi hi hai.

Yeh teen jagah ho sakti hai: **Code restructuring** (code ko saaf karna), **Data restructuring** (data ka design saaf karna), aur **Architecture restructuring** (poore system ke hisson ke rishte saaf karna, jaise UI, logic aur database ko alag layers mein rakhna).

Hum restructuring kyun karte hain? Taake software ko **samajhna aur badalna aasan** ho, bugs kam hon, aur aage reengineering ya naye features ke liye tayyari ho. Ek khaas baat: yeh **same level par** hoti hai, yaani code se code, design se design.

Paper mein aise likho:
Software restructuring is the improvement of a system's internal structure without changing its external behavior. It can be applied to code, data and architecture. Its purpose is better understandability, easier maintenance and fewer defects.

Topic 2: Restructuring ki Definition

Yeh exam ke liye ek seedhi definition hai. Asaan lafzon mein: **software ko andar se nayi, behtar shakal dena, magar bahar se kaam wohi rakhna.** Log aksar isay sirf 'code saaf karna' samajhte hain, jo ek had tak theek hai, magar restructuring data aur architecture par bhi ho sakti hai.

Example: Do kamray ka ghar. Saman uljha hua pada tha, aap ne tarteeb se rakh diya. Ghar wohi hai, functionality wohi hai, bas rehna aasan ho gaya. Yehi restructuring hai.

Paper mein aise likho:
Restructuring is the transformation of a system from one representation to another at the same abstraction level, while preserving its external behavior (Chikofsky and Cross).

Topic 3: Code Restructuring ki Definition

Purane programs mein aksar **spaghetti code** hota hai: GOTO ka jaal, ek 'if' ke andar doosra 'if' phir teesra, aur ek bohat lamba function. Code restructuring mein hum isay aisi shakal dete hain jo **parhne aur badalne mein aasan** ho. Kaam wohi rehta hai.

Hum kya kya karte hain: GOTO ko loops ya if se badalte hain; lambe code ko chhote functions mein todte hain; nested if ko saaf karte hain; bina matlab ke numbers (magic numbers) ko naam wale constants dete hain; woh code hatate hain jo kabhi chalta hi nahi (dead code); aur naam saaf rakhte hain.

Pehle (nested ifs, parhna mushkil):

```
def fee(student):
    if student.type == "regular":
        if student.scholarship:
            if student.marks > 80:
                return 0
            else:
                return 5000
        else:
            return 10000
    else:
        return 12000
```

Baad mein (restructuring ke baad, result bilkul wohi):

```
def fee(student):
    if student.type != "regular":
        return 12000
    if not student.scholarship:
        return 10000
```

```
    if student.marks > 80:
        return 0
    return 5000
```

Dekha? Pehle code teen ghere (nesting) ke andar tha. Ab har shart ek ke baad ek seedhi line mein hai. Result wohi hai, magar ab parhna bohat aasan hai.

Paper mein aise likho:
Code restructuring is modifying the source code to improve its structure and readability without changing its external behavior. It typically converts unstructured, spaghetti code into clean, modular code by removing GOTOs, deep nesting and dead code.

Topic 4: Refactoring

Refactoring ka matlab hai code ko **chhote chhote mehfooz qadam** mein andar se saaf karna, aur har qadam ke baad **tests chala kar** dekhna ke kuch toota to nahi. Behavior bilkul nahi badalta. Is ka mashhoor naam Martin Fowler hai jis ne isay phailaya.

Refactoring ke 4 usool yaad rakho: (1) bohat chhote qadam, ek ek kar ke; (2) pehle se tests honi chahiye aur har qadam ke baad chalani hain; (3) naya feature aur refactoring ek saath nahi, alag alag; (4) behavior badalna mana hai.

Refactoring aur Restructuring mein farq: restructuring ek bara concept hai jo code, data aur architecture sab par lagu hota hai, aksar planned bara kaam hota hai. Refactoring zyada tar **code level par chhote qadam** hain jo developer roz karta rehta hai. Dono mein behavior nahi badalta.

Aam refactoring techniques: Extract Method (code ke tukre ka naya function banana), **Extract Class** (bari class ko do mein todna), **Rename** (naam saaf karna), **Move Method** (method ko us class mein bhejna jahan woh belong karta hai), **Inline Method** (bohat chhote bekar function ko seedha code mein daal dena), **Replace Magic Number with Constant** (86400 ki jagah SECONDS_IN_DAY), aur **Pull Up Method** (do subclasses ka same method parent mein le jana).

Faida: code samajhna aasan, bugs kam, naye features aasan, duplicate hat jata hai. **Khatra:** bina tests ke refactor karoge to bug aa sakta hai.

Paper mein aise likho:
Refactoring is a disciplined technique of restructuring existing code by making small changes that improve its internal structure without changing its external behavior (Fowler). It is done in small steps with tests after each step. Common techniques are Extract Method, Extract Class, Rename and Move Method.

Topic 5: Code Smell

Jab khana kharab ho jata hai to bu aati hai. Bu khud masla nahi, magar ek **nishani** hai ke dhyan do. Code mein bhi aisi nishaniyan hoti hain jinhein **code smell** kehte hain: yeh batati hain ke yahan design ka masla ho sakta hai, chahe code abhi chal raha ho. Smell ka matlab bug nahi, balke 'is jagah ko refactor karo'.

Aam smells aur unka hal:

1) Long Method: bohat lamba function jo bohat kaam karta hai. Hal: Extract Method, usay chhote functions mein todo.

2) Large Class: class mein bohat zyada fields aur methods, kayi zimmedariyan. Hal: Extract Class.

3) Duplicate Code: wohi code kayi jagah. Hal: usay ek function mein jama karo.

4) Long Parameter List: function ko bohat zyada parameters. Hal: parameters ko ek object mein jama karo.

5) Feature Envy: ek class ka method doosri class ka data zyada use karta hai (jaise doosre ke ghar ka khana khana). Hal: method ko us doosri class mein le jao.

6) Shotgun Surgery: ek chhote change ke liye kayi classes mein tabdeeli karni parti hai. Hal: related code ek jagah jama karo.

7) Divergent Change: ek hi class kayi alag alag wajahon se badalti hai. Hal: usay alag classes mein todo.

8) Switch / lambi if chain: type ke mutabiq baar baar lambi switch. Hal: polymorphism.

9) Dead Code: woh code jo kabhi chalta hi nahi. Hal: hata do.

10) Magic Numbers: bina matlab ke numbers seedhe code mein, jaise 0.17. Hal: naam wala constant, TAX_RATE.

11) Data Clumps: wohi 3-4 variables hamesha ek saath aate hain. Hal: unki ek class bana do.

12) Comments as deodorant: kharab code ko samjhane ke liye bohat zyada comments. Hal: code hi saaf likho, naam theek rakho.

Paper mein aise likho:
A code smell is a surface symptom in the source code that usually indicates a deeper design problem. It is not a bug, but a sign that the code should be refactored. Common smells are Long Method, Large Class, Duplicate Code, Long Parameter List, Feature Envy and Shotgun Surgery.

Topic 6: Function Extraction (Extract Method)

Jab ek function bohat lamba ho jata hai aur andar kayi alag alag kaam kar raha hota hai, to hum us ke andar ka ek **tukra** alag **naye function** mein nikaal dete hain, usay ek matlab wala naam dete hain, aur purani jagah us naye function ko **call** kar dete hain. Isay **function extraction** ya **Extract Method** kehte hain. Yeh 'Long Method' smell ka sab se aam hal hai.

Qadam: (1) woh tukra chuno jo ek kaam karta hai. (2) naya function banao aur uska naam aisa rakho jo batay ke woh kya karta hai. (3) tukra us mein daalo. (4) jo variables chahiye unhein parameters ya return se do. (5) purani jagah function call likho. (6) tests chalao.

Pehle (ek lamba function, andar teen kaam):

```
def print_invoice(order):
    print("=== INVOICE ===")
    total = 0
    for item in order.items:
        total += item.price * item.qty
    total = total + total * 0.17
    print("Total:", total)
```

Baad mein (har kaam ke liye alag function):

```
TAX_RATE = 0.17

def calculate_subtotal(order):
    return sum(i.price * i.qty for i in order.items)

def add_tax(amount):
    return amount + amount * TAX_RATE
```

```
def print_invoice(order):
    print("=== INVOICE ===")
    total = add_tax(calculate_subtotal(order))
    print("Total:", total)
```

Fayde: har function ek hi kaam karta hai. Function ka **naam** hi explanation ban jata hai (comments ki zaroorat kam). Wohi function kahin aur bhi use ho sakta hai. Alag alag test karna aasan. Aur tax rate badalna ho to sirf ek jagah badalna parta hai.

Paper mein aise likho:
Function extraction (Extract Method) means moving a fragment of code from a long function into a new function with a meaningful name and replacing the fragment with a call to it. It removes the Long Method smell, improves readability, avoids duplication and makes testing easier.

Topic 7: Class Extraction (Extract Class)

Jab ek class bohat saare kaam karne lag jaye (isay Large Class smell kehte hain), to us ke andar se **related fields aur methods ko ek nayi class** mein nikaal dete hain, aur purani class us nayi class ko use karti hai. Yeh usool **Single Responsibility** par based hai: **ek class, ek kaam**.

Kaise pata chale ke class extract karni chahiye? Class bohat bari ho gayi ho; kuch fields aur methods sirf ek khaas topic ke ho (jaise sirf address ke); ya class kayi alag wajahon se badalti ho.

Pehle (Employee class address ka bhi kaam kar rahi hai):

```
class Employee:
    name, salary
    street, city, postal_code
    def get_full_address(self): ...
    def calculate_pay(self): ...
```

Baad mein (address ki alag class nikal li):

```
class Address:
    street, city, postal_code
    def get_full_address(self): ...

class Employee:
    name, salary
    address = Address(...)
    def calculate_pay(self): ...
```

Ab Employee class sirf employee ka kaam karti hai, aur Address class kahin aur bhi (jaise Customer mein) use ho sakti hai.

Qadam: (1) alag zimmedari pehchano; (2) nayi class banao; (3) related fields aur methods usme move karo; (4) purani class mein nayi class ka object rakho; (5) saare references update karo; (6) tests chalao.

Paper mein aise likho:
Class extraction (Extract Class) means splitting a large class by moving related fields and methods into a new class. It follows the Single Responsibility principle, so each class does one job. It makes classes smaller, easier to understand and test, and allows reuse.

Topic 8: Duplicate Code

Duplicate code ka matlab hai **wohi ya milta julta code kayi jagah dohraya hua**. Zyada tar yeh **copy-paste** ki wajah se hota hai. Yeh sab se mashhoor code smell hai.

Yeh bura kyun hai? Maan lo ek bug mila. Aap ne ek jagah theek kiya, magar wohi code 5 aur jagah bhi tha aur woh waisa hi ghalat reh gaya. Ya discount ka rate badalna ho to har jagah badalna parega, aur ek bhi bhool gaye to alag jagah alag discount. Is se code bhi bara hota hai aur maintenance mehnga.

Example: Teen jagah discount calculate karne ka wohi 5 line ka code copy kiya hua hai. Rate badalna ho to teeno jagah badalna parta hai. Ek jagah bhool gaye to wahan purana discount lagta rahega.

Code clones ki 4 qismein: Type 1: bilkul same code (sirf spaces ya comments ka farq). **Type 2:** structure same, magar naam ya values alag. **Type 3:** copy karke thori tabdeeli (kuch lines add ya delete). **Type 4:** code alag likha hai magar kaam bilkul wohi.

Hatane ke tareeqe: Extract Method (common code ko ek function mein), **Pull Up Method** (do subclasses ka same code parent mein), **Extract Class**, aur **Template Method** (steps same, sirf ek hissa alag). Tools jaise SonarQube aur PMD CPD duplicate code dhoondne mein madad karte hain.

Pehle:

```
def student_discount(price):
    return price - price * 0.10

def teacher_discount(price):
    return price - price * 0.10
```

Baad mein (ek hi function):

```
def apply_discount(price, rate=0.10):
    return price - price * rate
```

Paper mein aise likho:
Duplicate code is the same or very similar code repeated in several places, usually through copy-paste. It increases maintenance cost and the risk of inconsistent bug fixes. It is removed with refactoring such as Extract Method, Pull Up Method and Template Method. Clones are classified as Type 1 to Type 4.

Topic 9: Code Reliability Improved

Reliability ka matlab hai software par **bharosa kiya ja sake**: baar baar, har haalat mein, bina crash aur bina ghalti ke sahi kaam kare. Restructuring aur refactoring se code **saaf aur chhota** hota hai. Saaf code mein bugs ko chhupne ki jagah kam milti hai, is liye reliability barhti hai.

Reliability kaise improve hoti hai (wajah ke saath):

1) Code saaf aur chhota hone se: parhna aasan, to ghalti jaldi nazar aati hai. **2) Duplicate code hatane se:** ek bug ka ek hi jagah ilaaj, baqi jagah purana bug nahi rehta. **3) Dead aur bekar code hatane se:** kam code, to bug chhupne ki kam jagah. **4) Complexity kam karne se:** kam raaste (paths), to kam galat nateeje, aur test karna aasan. **5) Behtar error handling:** ghalat input ya error par app crash na ho (try/except, input check). **6) Tests ke saath refactoring:** har tabdeeli ke baad tests, is se purani cheezein tootti nahi (regression nahi aata). **7) Coupling kam, cohesion zyada:** ek jagah ka change doosri jagah nahi tora. **8) Static analysis tools:** code chalaye baghair hi masle pakar lete hain.

Reliability kaise naapte hain: defect density (kitne bugs per 1,000 lines), **MTBF** (do failures ke beech ka average waqt, zyada achha), failure rate, test coverage aur cyclomatic complexity (kam ho to behtar).

Example: Ek 200 line ka function tha jis mein tax ka hisaab teen jagah alag alag tha aur ek jagah ghalat. Function extraction se tax ka hisaab ek hi function mein aaya, jahan test bhi likhi gayi. Ab tax hamesha sahi aata hai. Yehi reliability barhna hai.

Paper mein aise likho:
Code reliability is improved by restructuring and refactoring because the code becomes cleaner, smaller and less complex. Removing duplicate and dead code, reducing complexity, adding error handling and unit tests, and lowering coupling all reduce defects. Reliability can be measured with defect density and mean time between failures (MTBF).

Aakhir mein: Quick Yaad Dehani

Topic	Yaad rakho
Legacy system	Purana, zaroori, mushkil se badalne wala software. 4 options: Scrap, Maintain, Reengineer, Replace.
Reengineering	Purane system ko andar se dobara tayyar karna, kaam wohi. Reverse + Restructure + Forward.
Maintenance ki 4 qismein	Corrective, Adaptive, Perfective, Preventive.
Lehman ke laws (8)	Continuing change, Increasing complexity, Self-regulation, Org. stability, Familiarity, Continuing growth, Declining quality, Feedback system.
Process model (6)	Inventory analysis, Document restructuring, Reverse engineering, Code restructuring, Data restructuring, Forward engineering.
Restructuring vs Refactoring	Dono mein behavior nahi badalta. Restructuring bara (code, data, architecture); refactoring chhote code-level qadam.
Extract Method / Class	Lambe function ya bari class ko todna.
Reliability	Saaf code + kam duplicate + tests + error handling = kam bugs.

Paper tip: har sawal mein pehle **definition**, phir **2-3 points ya explanation**, phir **ek example** likho. Code wale topics (restructuring, extract method, extract class, duplicate code) mein chhota before/after code zaroor likho, examiner ko bohat pasand aata hai.

Allah kare mids behtareen jayein, Moto! Agar sir ki slides mil jayein to bhej dena, main notes un ke mutabiq milaan kar dunga.